	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering** Department of Information Technology
Assignment 8	

NAME -	Viraj Pathare
PRN -	124B1F149
CLASS -	SY - IT
DIV -	A

Aim: Aggregation and Indexing: Design and Develop Queries using aggregation and indexing with suitable example using Mongo DB.

Theory:

Q1. Explain Aggregation in MongoDB with its stages.

Answer:

Aggregation in MongoDB is a framework used to **process and analyze data** by performing operations such as filtering, grouping, sorting, and calculating results. It works using an **aggregation pipeline**, where documents pass through multiple stages and are transformed step by step.

Common stages of aggregation:

- $match → Filters documents (similar to WHERE in SQL)
- $group → Groups data and performs operations like SUM, AVG, COUNT
- $sort → Sorts the result
- $project → Selects required fields
- $lookup → Performs join between collections

Q2. What is Indexing in MongoDB? Explain its types and advantages.

Answer:

Indexing in MongoDB is a technique used to **improve the performance of queries** by creating a data structure that allows faster searching. Without indexing, MongoDB performs a **collection scan**, which is slow.

Types of Indexes:

- **Single Field Index** → Index on one field
- **Compound Index** → Index on multiple fields
- **Multikey Index** → For array fields
- **Text Index** → For text search

Example:

db.employees.createIndex({ salary: 1 })

This creates an index on salary field (ascending order).

Advantages:

- Faster query execution
- Reduces search time
- Improves performance of large databases

Disadvantages:

- Uses extra memory
- Slows down insert/update operations slightly

Q3. Explain the difference between Aggregation and Indexing in MongoDB.

Answer:

Feature	Aggregation	Indexing
Purpose	Data processing & analysis	Improve query speed
Usage	$group, $match, $sort	createIndex()

Function Performs calculations Optimizes searching

Example Average salary Faster salary search

Aggregation is used to **analyze data**, while indexing is used to **optimize performance**.

Implementation:

Step 1: Create Database & Collection

Step 2: INSERT (Create Operation)

```
mongosh mongodb://127.0.0.

test> use CompanyDB
switched to db CompanyDB
CompanyDB>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
CompanyDB> db.createCollection("employees")
{ ok: 1 }
CompanyDB> db.employees.insertMany([
| { eid: 1, name: "Amit", salary: 50000, department: "IT" },
| { eid: 2, name: "Neha", salary: 40000, department: "HR" },
| { eid: 3, name: "Raj", salary: 30000, department: "Finance" },
| { eid: 4, name: "Simran", salary: 45000, department: "IT" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69de7a4c9b2d58ddc63682d1'),
    '1': ObjectId('69de7a4c9b2d58ddc63682d2'),
    '2': ObjectId('69de7a4c9b2d58ddc63682d3'),
    '3': ObjectId('69de7a4c9b2d58ddc63682d4')
  }
}
```

Aggregation Queries

1. Average Salary by Department

2. Count Employees in Each Department

```
CompanyDB> db.employees.aggregate([
| { $group: { _id: "$department", avgSalary: { $avg: "$salary" } } }
| ])
[ { _id: 'HR', avgSalary: 39000 }, { _id: 'IT', avgSalary: 52000 } ]
CompanyDB> db.employees.aggregate([
| { $group: { _id: "$department", totalEmployees: { $sum: 1 } } }
| ])
[ { _id: 'HR', totalEmployees: 2 }, { _id: 'IT', totalEmployees: 2 } ]
```

3. Employees with Salary > 40000 (Using $match)

```
CompanyDB> db.employees.aggregate([
| { $match: { salary: { $gt: 40000 } } }
| ])
[
  {
    _id: ObjectId('69de7a4c9b2d58ddc63682d1'),
    eid: 1,
    name: 'Amit',
    salary: 57000,
    department: 'IT'
  },
  {
    _id: ObjectId('69de7a4c9b2d58ddc63682d4'),
    eid: 4,
    name: 'Simran',
    salary: 47000,
    department: 'IT'
  }
]
```

4. Sort Employees by Salary (Descending)

5. Join + Aggregation (Employee with Projects)

mongosh mongodb://127.0.0.

```
  }
]
CompanyDB> db.employees.aggregate([
| {
| $lookup: {
| from: "projects",
| localField: "eid",
| foreignField: "eid",
| as: "project_details"
| }
| },
| { $project: { name: 1, salary: 1, project_details: 1 } }
| ])
[
  {
    _id: ObjectId('69de7a4c9b2d58ddc63682d1'),
    name: 'Amit',
    salary: 57000,
    project_details: [
      {
        _id: ObjectId('69de7a869b2d58ddc63682d5'),
        pid: 101,
        pname: 'Bank Management',
        eid: 1
      }
    ]
  },
  {
    _id: ObjectId('69de7a4c9b2d58ddc63682d2'),
    name: 'Neha',
    salary: 40000,
    project_details: [
      {
        _id: ObjectId('69de7a869b2d58ddc63682d6'),
        pid: 102,
        pname: 'E-Commerce',
        eid: 2
      }
    ]
  },
  {
    _id: ObjectId('69de7a4c9b2d58ddc63682d4'),
    name: 'Simran',
    salary: 47000,
```

Indexing Queries

1. Create Index on Salary

2. Create Compound Index (Department + Salary)

3. Check Indexes

4. Drop Index (Optional)

5. Use Index (Query Example)

```
CompanyDB> db.employees.createIndex({ salary: 1 })
salary_1
CompanyDB> db.employees.createIndex({ department: 1, salary: -1 })
department_1_salary_-1
CompanyDB> db.employees.dropIndex({ salary: 1 })
{ nIndexesWas: 3, ok: 1 }
CompanyDB> db.employees.find({ salary: 50000 })

CompanyDB>
```